

Analisi Semantica di TriaC

Gestione dello scoping

TriaC usa il singolo Scope per ridurre la complessità del back-end.

Regola di unicità globale: Ogni identificatore (sia esso il nome di una variabile o di un parametro formale di funzione) deve essere **globalmente unico** all'interno dell'intero programma.

La Symbol Table: Sarà rappresentata da un unico dizionario piatto (chiave-valore) in Python che memorizza le associazioni **Nome_Identificatore -> Tipo_Dato**.

Inferenza di tipo

TriaC **impone la tipizzazione esplicita** all'atto della dichiarazione. Questo garantisce che la Symbol Table sia popolata in modo deterministico e previene errori semantici di runtime.

Type Checking

Il Type System di TriaC introduce il meccanismo di **coercizione di tipo** tra i tipi numerici **int** e **double** :

- L'assegnazione o l'inizializzazione di un valore **int** a una destinazione **double** è consentita e genera un'istruzione LLVM **sitofp** (promozione a float a 64 bit).
- L'assegnazione o l'inizializzazione di un valore **double** a una destinazione **int** è consentita e genera un'istruzione LLVM **fpstoi** (troncamento a intero a 64 bit).

Schema di Implementazione

L'analisi semantica in TriaC viene eseguita tramite **due visite sequenziali** dell'AST (Visitor Pattern) :

- **Visita 1 (Raccolta Firme):** Il compilatore scorre l'albero per registrare i nomi e le firme di tutte le funzioni nella Symbol Table globale. Questo consente di supportare la chiamata a funzioni dichiarate più in basso nel sorgente (forward references).
- **Visita 2 (Type Checking):** Il compilatore esegue la visita approfondita di tutte le istruzioni ed espressioni applicando le azioni semantiche descritte di seguito.

Azioni Semantiche sui Nodi dell'AST

- **Azione A (Scoping - Semplificata):** All'avvio del compilatore viene inizializzata un'unica istanza di Symbol Table globale. Non vengono creati nuovi scope locali durante la visita di cicli o condizionali.
- **Azione B (Registrazione Identificatori - VarDeclNode, FunParamNode, FunNode):**
Quando si incontra una dichiarazione di variabile, parametro o funzione:
 - Se l'identificatore è già presente nella Symbol Table globale: solleva un errore semantico bloccante di **"dichiarazione multipla"**.
 - Altrimenti: inserisce l'identificatore e il rispettivo tipo nella Symbol Table.
- **Azione C (Riferimento a Identificatore - Identifier):**
Quando si incontra l'uso di una variabile o funzione in un'espressione:
 - Cerca l'identificatore nella Symbol Table globale.
 - Se non presente: solleva un errore semantico di **"identificatore non dichiarato"**.
 - Se presente: annota sul nodo AST il tipo associato.
- **Azione D (Costanti - LiteralNode):** Assegna direttamente al nodo il tipo corrispondente alla costante:
 - `IntegerConstantNode.type = int`
 - `DoubleConstantNode.type = double`
 - `BooleanConstantNode.type = boolean`
- **Azione E (Operatori ed Espressioni):** Verifica la compatibilità dei tipi dei nodi figli secondo le regole del Type System di TriaC. Se il controllo ha successo, assegna al nodo padre il tipo risultante, altrimenti solleva un **"errore di tipo"**.

Type System

Di seguito sono definite le regole formali di tipizzazione e compatibilità per ciascun nodo dell'AST di TriaC :

3.1 Ciclo Iterativo (WhileNode)

- **Regola:** IF il tipo del nodo figlio `condizione` è valutato come `boolean` AND il blocco interno `BlockNode` ha tipo `NOTYPE` (ovvero non contiene errori semantici al suo interno). THEN il nodo `WhileNode` è semanticamente corretto e restituisce il tipo `NOTYPE`. ELSE restituisce `TypeError`.

3.2 Struttura Condizionale (IfStatNode)

- **Regola:** IF il tipo del nodo figlio `condizione` è valutato come `boolean` AND il blocco `then` e l'eventuale blocco `else` hanno tipo `NOTYPE`. THEN il nodo `IfStatNode` è semanticamente corretto e restituisce il tipo `NOTYPE`. ELSE restituisce `TypeError`.

3.3 Assegnazione (AssignNode)

- **Regola:** IF l'identificatore di destinazione `ID` e il nodo espressione `ExprRef` a destra dell'operatore `=` soddisfano una delle seguenti condizioni di compatibilità :
 1. `ID.type == ExprRef.type`

2. `ID.type == double AND ExprRef.type == int` (genera un nodo implicito di cast `sitofp`)
3. `ID.type == int AND ExprRef.type == double` (genera un nodo implicito di cast `fptosi`) **THEN** l'assegnazione è valida e restituisce il tipo **NOTYPE**. **ELSE** restituisce **TypeError** (es. tentativo di assegnare un booleano a un intero).

3.4 Operazioni Aritmetiche Binarie (**BinOpNode** - Add, Sub, Mul, Div)

- **Regola:**

IF i tipi dei due nodi figli soddisfano una delle seguenti condizioni:

1. Entrambi sono di tipo **int** **THEN** il nodo risultante ha tipo **int**.
2. Entrambi sono di tipo **double** **THEN** il nodo risultante ha tipo **double**.
3. Uno è **int** e l'altro è **double** **THEN** l'operando di tipo **int** viene promosso a **double** tramite cast implicito e il nodo risultante ha tipo **double**. **ELSE** restituisce **TypeError**.

3.5 Operazioni di Confronto (**BinOpNode** - ==, !=, <, >, <=, >=)

- **Regola:**

IF i due operandi sono entrambi di tipo numerico (**int** o **double**) . **THEN** l'operazione è consentita (applicando la promozione a **double** se uno dei due è **int**) e il tipo finale del nodo è **boolean**. **ELSE** restituisce **TypeError**.

3.6 Operazioni Logiche (**BinOpNode** - And, Or, Not)

- **Regola:**

IF gli operandi sono rigorosamente di tipo **boolean**. **THEN** l'operazione è corretta e il tipo risultante del nodo è **boolean**. **ELSE** restituisce **TypeError**.

3.7 Chiamata a Funzione (**FunCallNode**)

- **Regola:**

IF l'identificatore della funzione è presente nella Symbol Table globale **AND** il numero di argomenti passati corrisponde al numero di parametri dichiarati nella firma della funzione **AND** il tipo di ciascun argomento passato è compatibile (anche tramite cast implicito) con il tipo del rispettivo parametro formale. **THEN** la chiamata è semanticamente corretta e il nodo assume il tipo di ritorno dichiarato della funzione. **ELSE** restituisce **TypeError**.

3.8 Istruzione di Ritorno (**ReturnNode**)

- **Regola:**

IF il tipo dell'espressione restituita **ExprRef** è compatibile (anche tramite cast implicito) con il tipo di ritorno dichiarato nella firma della funzione corrente . **THEN** l'istruzione è corretta e restituisce il tipo **NOTYPE**. **ELSE** solleva un errore semantico di firma non rispettata.

3.9 Controllo dell'Entry Point del Programma (**main**)

- **Regola:**

Al termine dell'analisi semantica, l'analizzatore deve verificare che nella Symbol Table globale esista una funzione registrata con la firma esatta:
`func main() -> int`

- Se la funzione non esiste o ha parametri o ha un tipo di ritorno diverso da `int`, il compilatore blocca l'esecuzione segnalando la mancanza del punto di ingresso.